



Helm & Templating

Package Management for Kubernetes — Module 12 of
13

Enterprise EKS Platform





Agenda

Core Concepts

- Why Helm — the problem it solves
- Charts, releases, and repositories
- Chart structure and values
- Go template language

Operations & Advanced

- Install, upgrade, rollback lifecycle
- Hooks, dependencies, and testing
- Helm vs Kustomize
- Best practices & Lab 12

Why Helm?

The problem with raw YAML at scale

The YAML Sprawl Problem

A typical microservice requires 5–10 YAML manifests —
Deployment, Service, ConfigMap, Secret, Ingress, HPA, PDB,
ServiceAccount, NetworkPolicy...

- **Duplication** — same patterns repeated across dozens of services
- **Environment drift** — dev, staging, and prod configs diverge over time
- **No versioning** — hard to track what's deployed or roll back atomically
- **No dependency management** — ordering and relationships are manual
- **Parameterization** — sed and envsubst don't scale

Helm Core Concepts

Chart

A package of pre-configured Kubernetes resources — templates + default values + metadata

Release

A running instance of a chart in a cluster — each install creates a new release with its own history

Repository

A server hosting packaged charts — similar to Docker Hub for images or npm for packages

Key relationship: A *chart* is installed into a cluster as a *release*. The same chart can be installed multiple times, each producing a distinct release.



Helm Architecture

Chart structure, values, and the rendering pipeline

Chart Directory Structure

```
mychart/
├── Chart.yaml           # Chart metadata (name, version, dependencies)
├── Chart.lock           # Locked dependency versions
├── values.yaml          # Default configuration values
├── values.schema.json   # Optional JSON Schema for values
├── templates/           # Kubernetes manifest templates
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── _helpers.tpl     # Reusable named templates
│   ├── NOTES.txt        # Post-install usage instructions
│   └── tests/
│       └── test-connection.yaml
├── charts/              # Packaged dependency charts
└── crds/                # Custom Resource Definitions
```

Chart.yaml is the only required file — it defines the chart's API version, name, and version.

Values and Override Hierarchy

Helm merges values from multiple sources in precedence order:

Override Precedence (lowest to highest)

1. **Parent chart's values.yaml** — defaults from dependency charts
2. **Chart's values.yaml** — the chart's own defaults
3. **-f custom-values.yaml** — user-supplied values files
4. **--set key=value** — command-line overrides (highest priority)

```
# Override with a file and individual values
helm install my-app ./mychart \
  -f env-values.yaml \
  --set image.tag=v2.1.0
```




Templating

Go templates, functions, and control flow

Go Template Basics

```
metadata:  
  name: {{ .Release.Name }}-app  
  labels:  
    app.kubernetes.io/name: {{ .Chart.Name }}  
spec:  
  replicas: {{ .Values.replicaCount }}  
  # ...  
  containers:  
    - image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

Double curly braces `{{ }}` delimit actions. The dot (`.`) is the root context object.

Built-in Objects

.Values

Contents of `values.yaml`
merged with overrides

.Chart

Contents of `Chart.yaml` —
`.Name`, `.Version`,
`.AppVersion`

.Release

`.Name`, `.Namespace`,
`.IsInstall`, `.IsUpgrade`,
`.Revision`

.Capabilities

`.KubeVersion`,
`.APIVersions` — detect cluster
features at render time

Tip: Use `.Capabilities.APIVersions.Has` to conditionally include

Template Functions

Helm includes 60+ functions from Go templates and the Sprig library:

```
# default - provide fallback values
image: {{ .Values.image.tag | default "latest" }}

# quote - wrap in double quotes
env:
  - name: APP_ENV
    value: {{ .Values.env | quote }}

# required - fail if value is missing
namespace: {{ required "namespace is required" .Values.namespace }}

# toYaml + nindent - output structured YAML
resources:
{{ toYaml .Values.resources | nindent 10 }}

# upper, lower, trim, replace
name: {{ .Values.name | lower | trunc 63 | trimSuffix "-" }}
```

Pipeline syntax (|) chains functions left to right, passing output as the last argument to the next function.

Control Structures

```
# if/else - conditional blocks
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ .Release.Name }}-ingress
{{- end }}

# range - iterate over lists and maps
{{- range .Values.extraEnvVars }}
  - name: {{ .name }}
    value: {{ .value | quote }}
{{- end }}

# with - change scope
{{- with .Values.nodeSelector }}
nodeSelector:
  {{- toYaml . | nindent 2 }}
{{- end }}
```

Named Templates & _helpers.tpl

Define in _helpers.tpl

```
{{- define "mychart.labels" -}}  
app.kubernetes.io/name: {{ .Chart.Name }}  
app.kubernetes.io/instance: {{ .Release.Name }}  
app.kubernetes.io/version: {{ .Chart.AppVersion }}  
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}  
{{- end }}
```

Use in templates

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: {{ .Release.Name }}-app  
  labels:  
    {{- include "mychart.labels" . | nindent 4 }}
```

Use `include` over `template` — `include` captures output as a string so you can pipe it through functions like `nindent`.



Helm Operations

Install, upgrade, rollback, and release management

Release Lifecycle Commands

```
# Install a chart as a new release
helm install my-release ./mychart -n myapp

# Upgrade (creates new revision)
helm upgrade my-release ./mychart -f env-values.yaml

# Install or upgrade in one command (idempotent)
helm upgrade --install my-release ./mychart

# Rollback to a previous revision
helm rollback my-release 2

# Uninstall a release
helm uninstall my-release -n myapp
```

Best practice: Always use `helm upgrade --install` in CI/CD pipelines for idempotent deployments.

Repository Operations

```
# Add a chart repository and update cache
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Search for charts
helm search repo nginx
helm search repo bitnami/postgresql --versions

# Download a chart without installing
helm pull bitnami/postgresql --version 12.5.0 --untar
```

helm pull is useful for vendoring charts into your repository or inspecting their templates before deployment.

Debugging & Validation

helm template

```
# Render templates locally
helm template my-release ./mychart

# Render with overrides
helm template my-release ./mychart \
  -f staging-values.yaml --set image.tag=v2.0

# Show only a specific template
helm template my-release ./mychart -s templates/deployment.yaml
```

helm lint

```
# Check chart for issues
helm lint ./mychart

# Lint with specific values; strict mode treats warnings as errors
helm lint ./mychart -f env-values.yaml --strict
```

Use in CI: `helm lint` + `helm template` form the first line of defense in chart validation.

Release Management & History

```
# List releases; show history
helm list --all-namespaces
helm history my-release

# Inspect a release
helm get values my-release      # user-supplied values
helm get manifest my-release    # rendered manifests
```

Release state is stored as Secrets in the release namespace. Each revision creates a new Secret named `sh.helm.release.v1.<name>.v<revision>`.



Advanced Helm

Hooks, dependencies, library charts, and testing

Helm Hooks

Hooks run at specific points in the release lifecycle:

```
metadata:
  name: {{ .Release.Name }}-db-migrate
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-weight": "1"
    "helm.sh/hook-delete-policy": hook-succeeded
# ... Job spec runs migration before upgrade
```

Common use cases: Database migrations before upgrade, cache warming after install, cleanup jobs before delete.

Helm Hook Types

Hook	When It Runs
pre-install	Before resources are created
post-install	After all resources are created
pre-upgrade / post-upgrade	Before/after upgrade
pre-rollback / post-rollback	Before/after rollback
pre-delete / post-delete	Before/after uninstall

hook-weight controls execution order when multiple hooks exist. **hook-delete-policy** controls whether hook resources are cleaned up automatically.

Declaring Dependencies

Chart.yaml

```
apiVersion: v2
name: my-application
version: 1.0.0
dependencies:
- name: postgresql
  version: "12.x"
  repository: "https://charts.bitnami.com/bitnami"
  condition: postgresql.enabled
- name: redis
  version: "17.x"
  repository: "https://charts.bitnami.com/bitnami"
  condition: redis.enabled
  alias: cache
```

condition and **tags** let users enable/disable dependencies. Use **alias** to install the same chart multiple times with different configs.

Managing Dependencies

```
# Download dependencies
helm dependency update ./mychart

# List dependencies
helm dependency list ./mychart
```

Override subchart values in values.yaml:

```
postgresql:
  enabled: true
  auth:
    username: myapp
    database: myapp_db
cache: # uses alias
  enabled: true
  architecture: standalone
```

Subchart values are scoped under the dependency name (or alias) in the parent's `values.yaml`.

Library Charts

What are Library Charts?

Charts that provide **only named templates** — no rendered manifests. They define common patterns shared across multiple application charts.

Creating

```
# Chart.yaml - type: library (no rendered manifests)
apiVersion: v2
name: common-lib
type: library
# ...

# templates/_deployment.tpl - reusable named template
{{- define "common.deployment" -}}
# ... standard Deployment with shared labels, security context, etc.
{{- end -}}
```

Consuming

```
# Chart.yaml dependency
dependencies:
- name: common-lib
  version: "1.x"
  repository: "@internal"
```

```
# templates/deployment.yaml
```

Helm Testing

Test Definition

```
# templates/tests/test-connection.yaml
metadata:
  name: {{ .Release.Name }}-test-connection
  annotations:
    "helm.sh/hook": test
# ... Pod spec that verifies connectivity
```

Running Tests

```
# Run tests after install/upgrade
helm test my-release
helm test my-release --logs # include pod logs in output
```

CI/CD integration: Include `helm test` in your pipeline after every deployment to catch issues before they reach users.

OCI Registry Support

Since Helm 3.8+, charts can be stored in OCI-compliant container registries alongside your container images.

```
# Package and push to OCI registry
helm package ./mychart
helm push mychart-1.0.0.tgz oci://registry.example.com/charts

# Install directly from OCI
helm install my-release oci://registry.example.com/charts/mychart \
--version 1.0.0
```

Advantages: Unified artifact storage, immutable versions, standard auth, no separate chart server needed.

Helm vs Kustomize

Feature	Helm	Kustomize
Approach	Templating (generate YAML)	Patching (overlay on base YAML)
Parameterization	Go templates + values	Strategic merge patches
Packaging	Charts with versioning	No packaging concept
Release tracking	Built-in (Secrets)	None (relies on GitOps)
Rollback	Built-in (<code>helm rollback</code>)	Via Git revert
Learning curve	Steeper (Go templates)	Moderate (YAML patches)
Built into <code>kubectl</code>	No (separate binary)	Yes (<code>kubectl -k</code>)

They complement each other. Many teams use Helm for packaging third-party charts and Kustomize for environment-specific overlays on top of `helm template` output.

Best Practices

Chart development guidelines and recommended patterns

Chart Development Guidelines

Naming & Structure

- Follow [Helm best practices guide](#)
- Use `helm create` to scaffold new charts
- Prefix template names with chart name
- Keep templates focused — one resource per file

Values & Defaults

- Provide sensible defaults for all values
- Use flat keys where possible (avoid deep nesting)
- Document every value with comments
- Never put secrets in `values.yaml`

Version your charts independently from your application. Bump the chart version for template changes, and `appVersion` for the application itself.

Values Schema: JSON Schema

values.schema.json

```
{
  "required": ["image", "service"],
  "properties": {
    "replicaCount": { "type": "integer", "minimum": 1 },
    "image": {
      "required": ["repository"],
      "properties": {
        "repository": { "type": "string" },
        "pullPolicy": { "enum": ["Always", "IfNotPresent", "Never"] }
      }
    }
  }
}
```

Values Schema: Validation Benefits

- **Type checking** — catch string vs number errors
- **Required fields** — fail fast on missing values
- **Range validation** — enforce min/max constraints
- **Enum validation** — restrict to allowed values
- **Pattern matching** — validate format (e.g., semver)

Validated automatically during `helm install`, `helm upgrade`, `helm lint`, and `helm template`.



Module Summary

- Helm is a package manager for Kubernetes using charts, templates, and values
- Go templates with values.yaml enable reusable, parameterized deployments
- Named templates in _helpers.tpl promote DRY patterns across resources
- Helm tracks release history enabling rollback to any previous revision
- OCI registry support stores charts alongside container images
- Helm and Kustomize complement each other for packaging and environment overlays



Lab 12: Helm and Templating

Estimated Time: ~45 minutes

- Install and explore a chart from the podinfo repository
- Upgrade a release and rollback to a previous revision
- Create a custom Helm chart using `helm create`
- Add custom templates with Go templating and values
- Validate with `helm lint`, `helm template`, and dry-run
- Package and install the custom chart

Key Takeaways

Remember These Points

1. Use `helm create` to scaffold charts and follow the official best practices guide
2. Version charts independently from applications and never put secrets in `values.yaml`
3. Add a `values.schema.json` for validation to catch configuration errors before deployment

? Questions & Discussion

Module 12: Helm & Templating

Up Next: Module 13 — CI/CD Integration & GitOps